TOPIC 2 — API Test Automation with GenAI

Relatório final de entrega | Automação REST API ServeRest

Aplicação: https://compassuol.serverest.dev/ | Data das evidências: 12/10/2026

Repositório público: github.com/arlensilvaair/API-Test-Automation-with-GenAI

Stack: Python 3.14.3, Pytest 9.0.2, Requests, pytest-html, JUnit XML. IA escolhida: ChatGPT.

1. Resumo verificável

Métrica	Resultado
Testes de integração coletados	30
Aprovados / falhas / erros / ignorados	30 / 0 / 0 / 0
Duração registrada em JUnit XML	39.383 segundos
Registro da execução	2026-10-12 11:41:40.783154-03:00
GitHub Actions	Workflow verde, conforme informação fornecida pelo participante; escopo do job não verificado neste relatório.

Interpretação: Todos os 30 casos executados passaram. Isso atesta o comportamento observado dos cenários automatizados naquele momento, sem demonstrar cobertura integral de todos os requisitos possíveis da aplicação.

2. Escopo coberto

• Autenticação: credenciais válidas e inválidas; payloads de login inválidos.

• Usuários: criação, consulta, duplicidade, listagem/filtros, atualização, exclusão e tipo inválido.

• Produtos: administração e autorização, duplicidade, filtros, atualização, exclusão e ID inexistente.

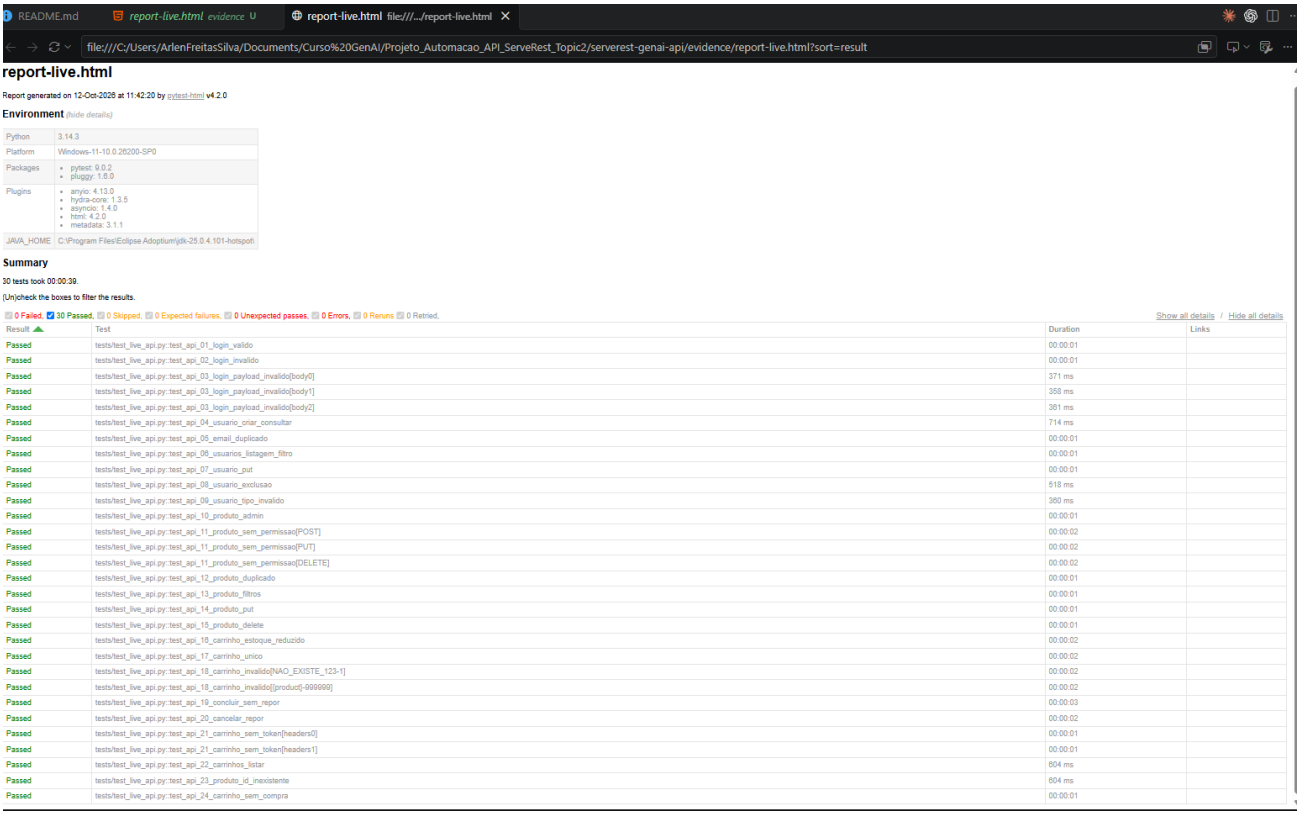
• Carrinhos: reserva e reposição de estoque, carrinho único, itens inválidos, conclusão/cancelamento, autenticação, consulta e compra vazia.

3. Condição de segurança e limite da evidência

A suíte foi executada com --run-live --insecure-live após uma falha de validação do certificado TLS. Assim, as requisições HTTP ocorreram sem verificação do certificado do servidor. O resumo do terminal apresentou 30 InsecureRequestWarning. Esta configuração deve ser restrita a ambiente de testes autorizado; não é recomendada para execução contínua nem para produção. Investigar e corrigir a cadeia de confiança TLS continua sendo uma ação recomendada.

4. Evidências da execução

Evidência A — Relatório HTML do pytest-html

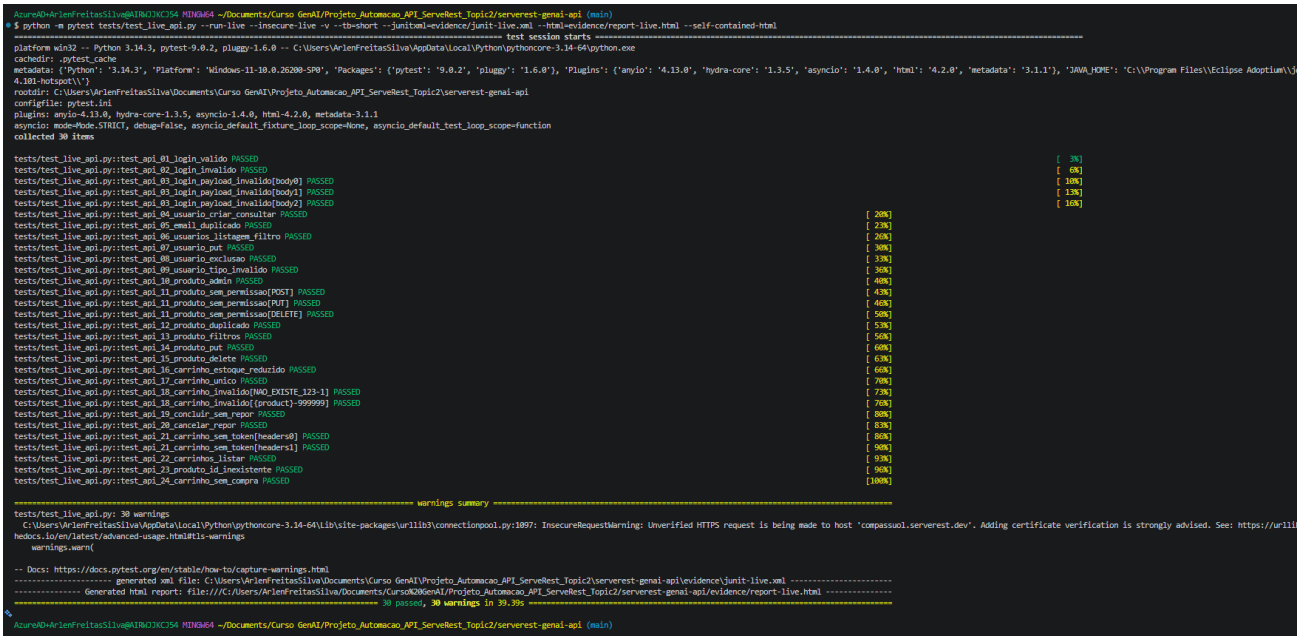


Captura fornecida: relatório visual com 30 Passed, 0 Failed, 0 Skipped e 0 Errors; lista detalhada dos casos.

Fonte estruturada: arquivo junit-live.xml, fornecido junto às capturas e inspecionado para confirmar totais, nomes e tempos.

4. Evidências da execução (continuação)

Evidência B — Terminal Pytest



Captura fornecida: execução com o comando de integração, 30 casos PASSED e resumo de 30 warnings em aproximadamente 39,39 s.

Comando utilizado:

python -m pytest tests/test_live_api.py --run-live --insecure-live -v --tb=short --junitxml=evidence/junit-live.xml --html=evidence/report-live.html --self-contained-html

5. Assistência de IA e revisão crítica

Ferramenta escolhida: ChatGPT

ChatGPT foi empregado no processo de planejamento e implementação da automação: transformação do plano de testes da Topic 1 em estrutura executável; sugestão de testes positivos, negativos e de borda; organização das fixtures e dados dinâmicos; configuração do relatório JUnit; apoio ao diagnóstico de erro SSL; e orientação para preparar evidências e documentar a entrega. A validação funcional ocorreu por execução real no ambiente do participante.

Registro de ajustes, verificações e decisão humana

ID	Ajuste / revisão	Decisão e evidência
01	Seleção da stack Adotado Python + Pytest + Requests para automação HTTP e relatórios.	Decisão de projeto.
02	Cenários e parametrização Suíte contém 24 identificadores lógicos (API-01 a API-24), desdobrados em 30 execuções por parametrização.	Conferido pelos nomes do XML.
03	Execução inicialmente bloqueada Requisições HTTPS falharam com SSLCertVerificationError, inclusive em fixtures.	Problema de ambiente/transporte, não defeito funcional demonstrado.
04	Alternativa operacional Execução com --insecure-live permitiu completar a suíte; 30 InsecureRequestWarning.	Aceito temporariamente para evidência de homologação, com ressalva explícita.
05	Validação de resultados Gerados XML JUnit e relatório HTML; capturas de tela mostram os mesmos 30 casos aprovados.	Conferência cruzada concluída.
06	Ponto de melhoria Resolver certificados CA/SSL para remover necessidade de --insecure-live.	Pendente, não bloqueia documentação dos resultados funcionais observados.

6. Rastreabilidade e pendências da entrega

Concluído: projeto executável, repositório GitHub indicado, execução local da API real, XML JUnit, captura do terminal, captura do relatório HTML, descrição do uso de GenAI e registro das correções de ambiente.

A confirmar pelo participante: fazer commit/push dos arquivos de evidência ao repositório, caso ainda não estejam publicados; conferir se o workflow GitHub Actions executa testes offline ou também testes live. A cor verde do workflow isoladamente não comprova execução live.

7. Arquivos-fonte das evidências

junit-live.xml • html-report.png.png • terminal-execution.png.png. Os resultados foram transcritos diretamente do XML enviado e cotejados visualmente com as capturas; nenhuma falha funcional foi omitida.